

Railway Reservation System

Individual Java-Based Desktop Application

Submitted By: Ujjwal Jalotra

Registration Number: 25BAI10118

Institution: VIT Bhopal University

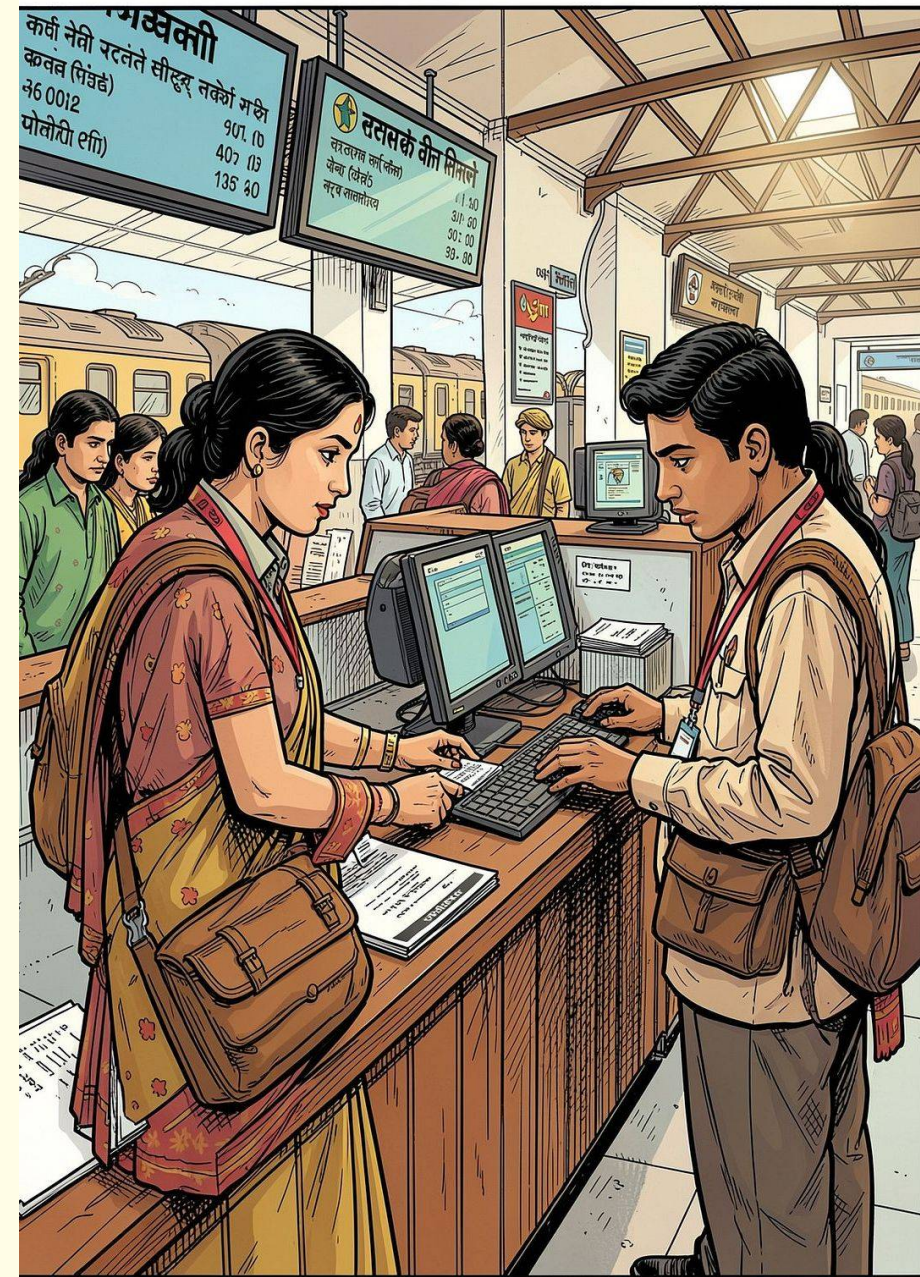
Introduction & Problem Statement

Introduction

The Railway Reservation System is a Java-based desktop application simulating core railway booking operations. It leverages OOP, Java Swing, and ArrayList collections to manage trains, passengers, and ticketing in a structured, object-oriented environment.

Problem Statement

Manual reservation processes are slow and error-prone. This project addresses the need for a structured digital system that manages train schedules, seat allocation, and passenger records — eliminating human error and improving operational efficiency.



Objectives & Requirements

Project Objectives

- Design a GUI-based reservation system using Java Swing
- Apply OOP principles — encapsulation, inheritance, polymorphism
- Manage train and passenger data using ArrayList collections
- Implement robust exception handling and input validation
- Simulate seat availability checks and ticket booking workflows

Functional Requirements

- Register, search, and manage train records
- Book and cancel passenger tickets
- Display seat availability in real time
- Generate booking confirmation details

Non-Functional Requirements

- Responsive Swing-based GUI with intuitive navigation
- Graceful error handling for invalid inputs
- Consistent in-memory data persistence per session

Technologies Used



Java & OOP

Core language with encapsulation, inheritance, and polymorphism for modular, reusable class design.



Java Swing

GUI toolkit providing buttons, text fields, tables, and dialog boxes for the desktop interface.



ArrayList Collections

Dynamic in-memory storage for managing trains, passengers, and booking records at runtime.

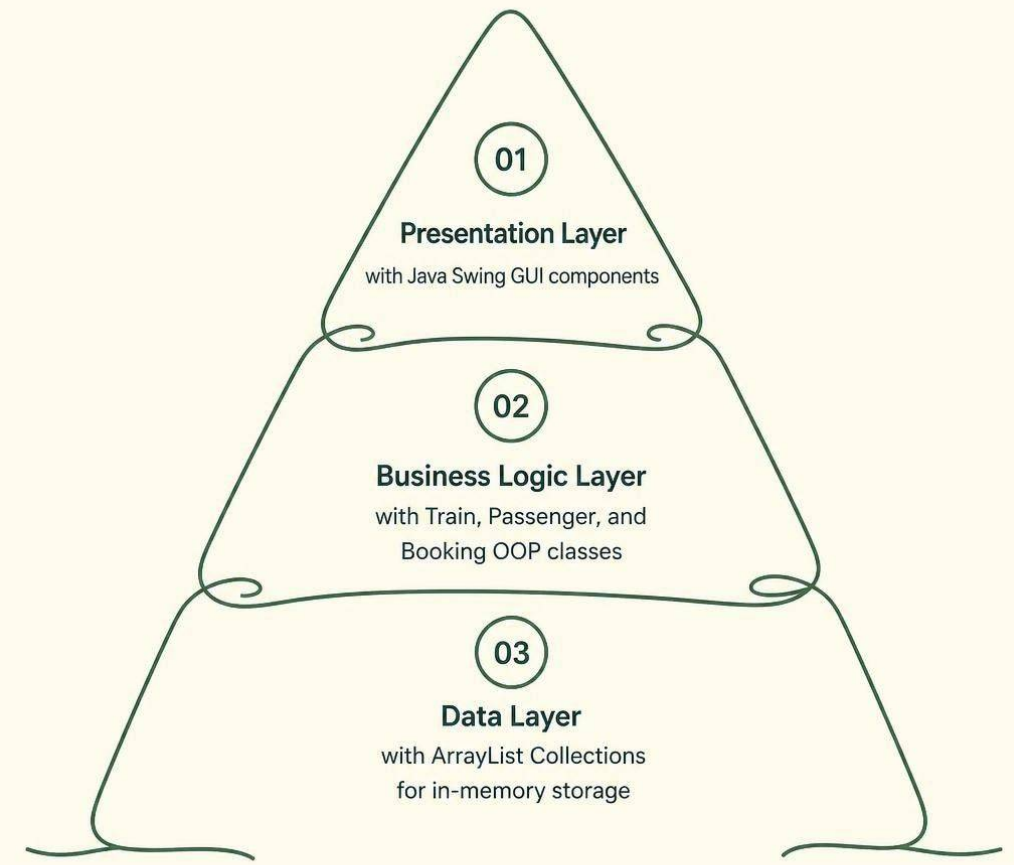


Exception Handling

Try-catch blocks and custom validation to handle runtime errors and invalid user inputs gracefully.

System Architecture

The system follows a layered architecture separating the **presentation layer** (Swing GUI), **business logic layer** (OOP classes), and **data layer** (ArrayList collections). This ensures clean separation of concerns and maintainability.



Presentation Layer

Java Swing — JFrame, JButtons, JTables, Dialogs

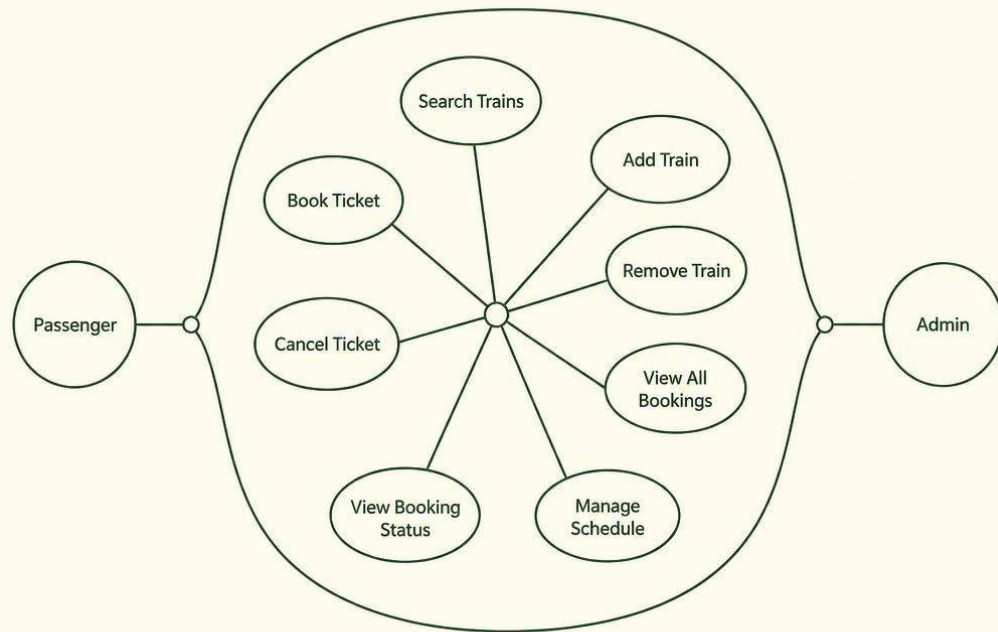
Business Logic Layer

Train, Passenger, Booking, and User classes

Data Layer

ArrayList-based in-memory storage

UML Diagrams — Use Case & Workflow



Use Case Overview

Two primary actors interact with the system. **Passengers** search trains, book or cancel tickets, and check booking status. **Admins** manage train records, schedules, and view all system bookings.

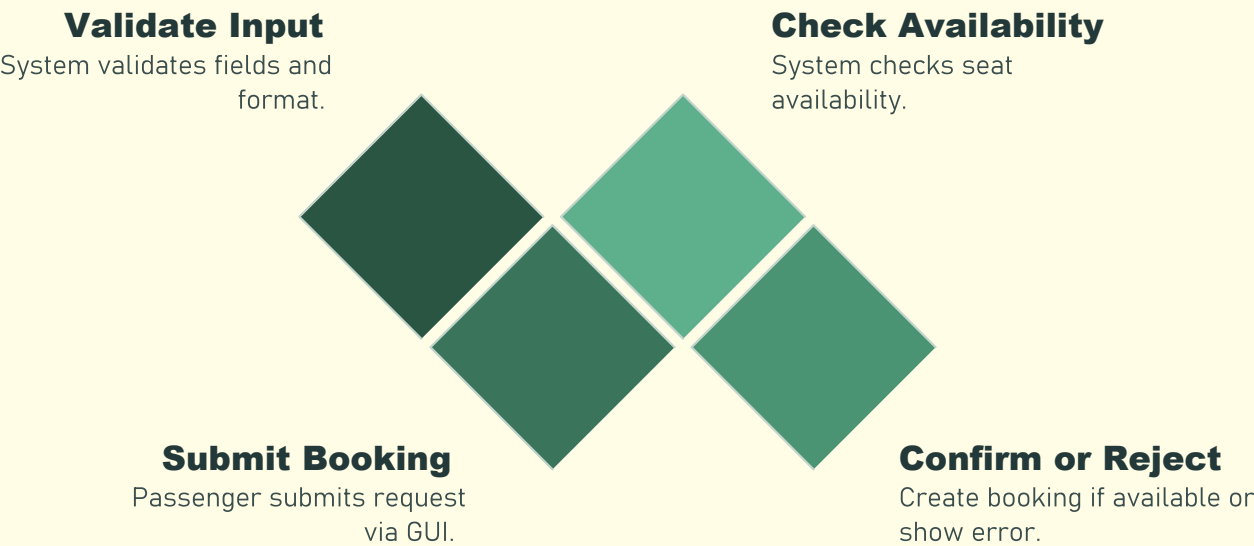
Booking Workflow

1. User launches application and logs in
2. Searches for trains by route or date
3. Selects a train and checks seat availability
4. Enters passenger details and confirms booking
5. System validates input and generates ticket

UML Diagrams — Sequence & Class

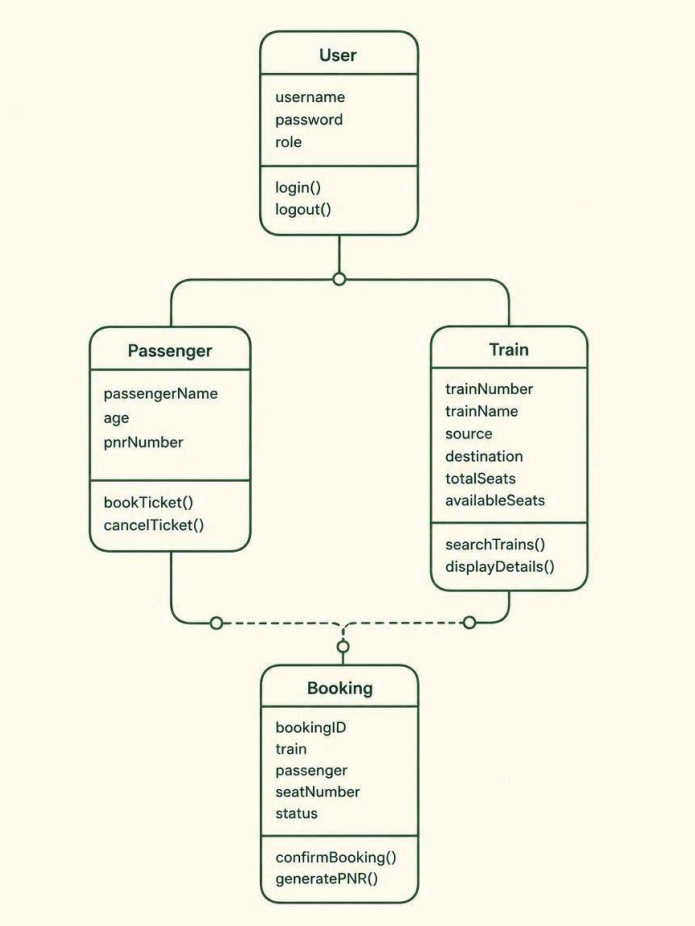
Sequence Diagram: Ticket Booking

Illustrates the interaction flow when a passenger books a ticket — from GUI input through validation, seat availability check, to confirmation or rejection.



Class Diagram

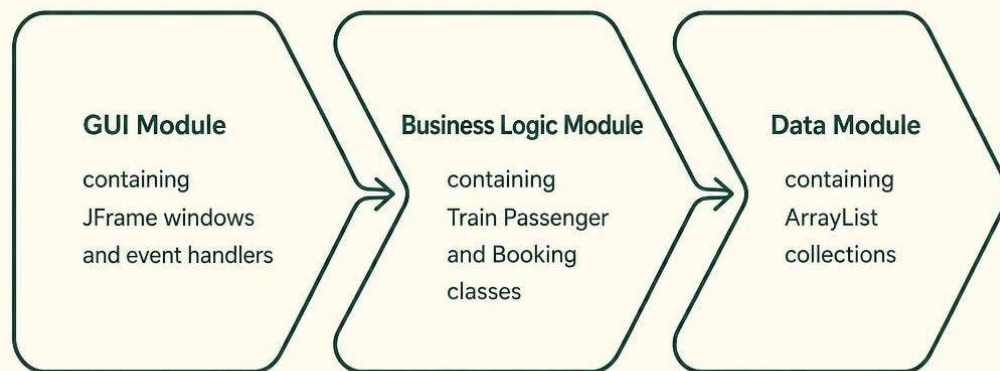
Core OOP structure with four primary classes and their relationships.



Component & ER Diagram

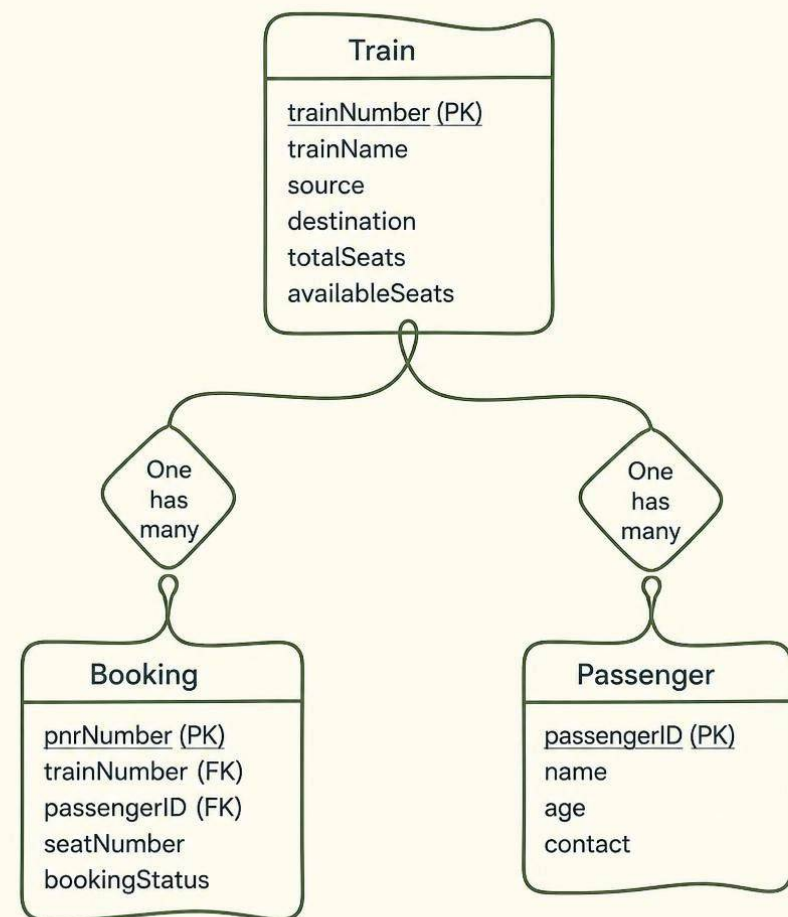
Component Diagram

The system comprises three main components: the **GUI Module** (Swing interfaces), the **Business Logic Module** (OOP classes), and the **Data Module** (ArrayList storage). Dependencies flow from GUI to Business Logic to Data.



ER Diagram (Logical Data Model)

Though data is stored in-memory, the logical entity relationships mirror a relational model for clarity and future database migration.



Implementation, Screenshots & Testing

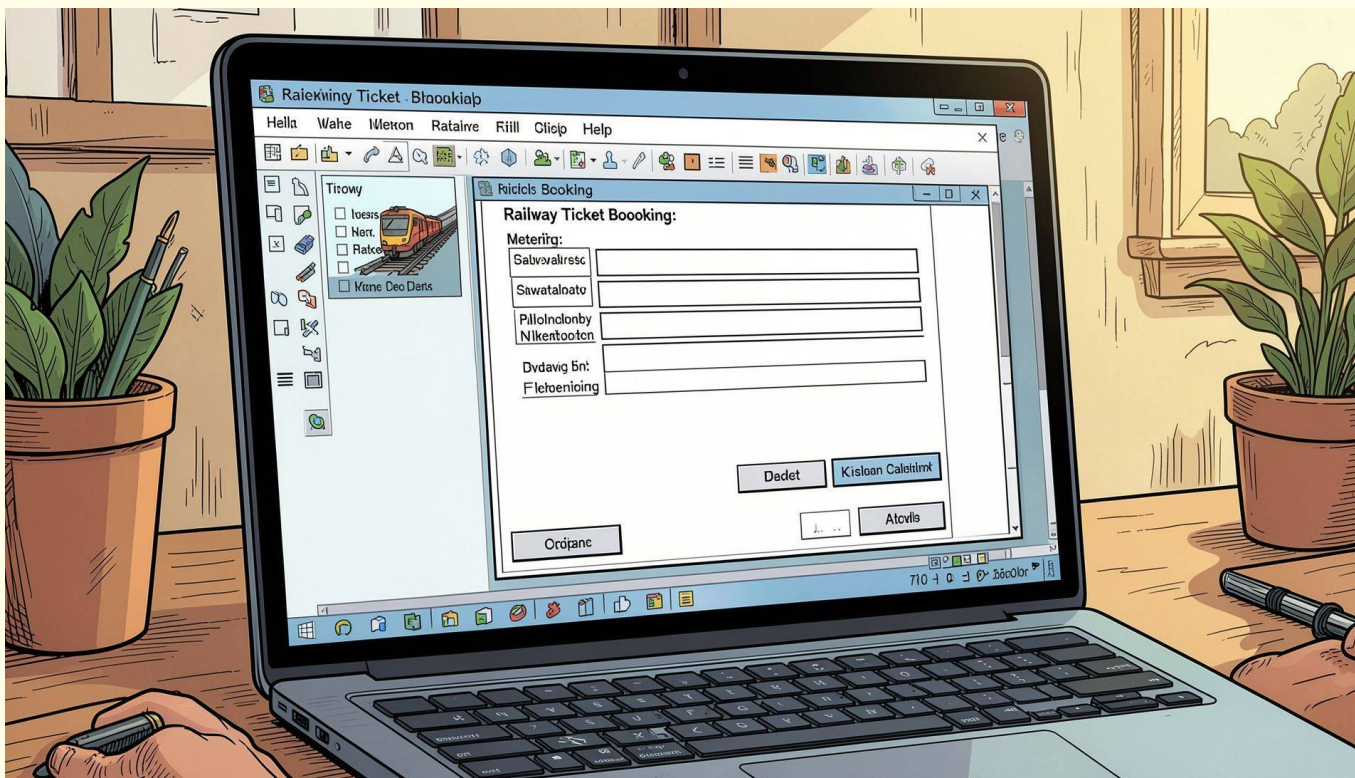
Key Implementation Highlights

- Modular class design with clear separation of concerns
- Swing event-driven architecture for GUI interactions
- PNR generation using timestamp-based logic
- Real-time seat count updates on booking/cancellation

Testing Approach

- **Unit Testing:** Individual class methods validated in isolation
- **Boundary Testing:** Edge cases for seat limits and date inputs
- **Manual Testing:** End-to-end GUI workflow verification
- **Exception Testing:** Invalid inputs and null-pointer scenarios

☐ All test cases were executed manually. No automated testing frameworks were used in this project.



Challenges, Learnings & Future Scope

Challenges Faced

- Managing concurrent seat allocation logic
- Designing intuitive Swing layouts
- Handling edge cases in input validation

Learning Outcomes

- Practical OOP application in a real-world context
- Java Swing GUI development and event handling
- Software design and structured problem-solving

Current Limitations

- Data is not persisted beyond the session
- No multi-user or network support
- Limited to single-route booking logic

Future Enhancements

- MySQL/JDBC integration for data persistence
- Multi-user login with authentication
- Payment gateway and e-ticket generation

